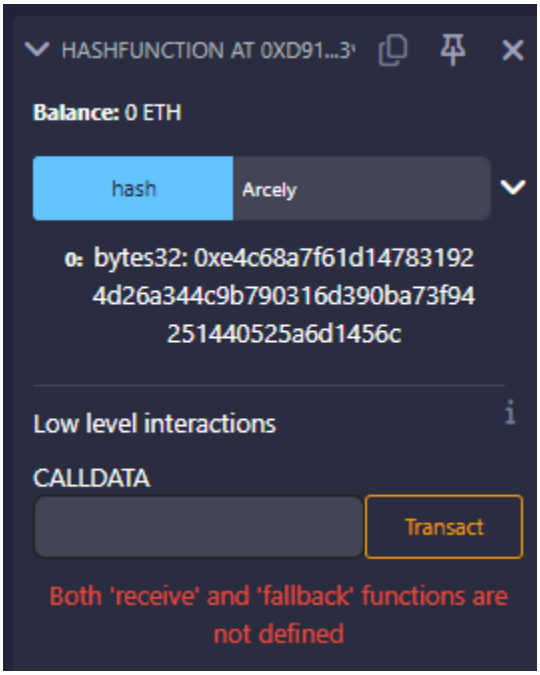
Opiniano, Ezkylle 01/27/2026

NW – 301 6BLOCKTECH

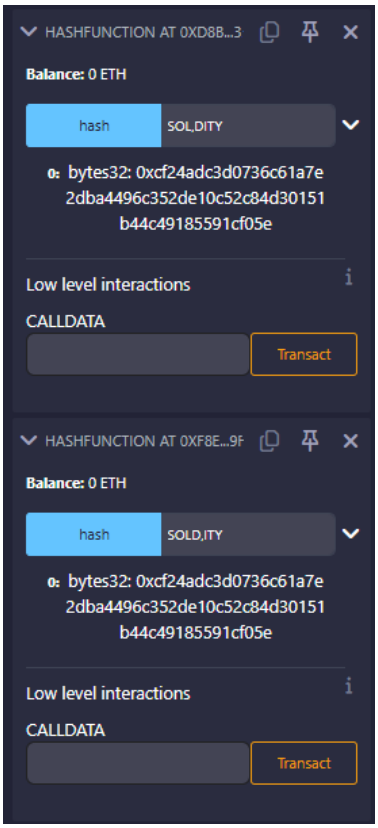
Midterm Lab Activity 4: Hash Function using Keccak256

```
function hash(string memory _text)    infinite gas
    public pure returns (bytes32)
    {
        return keccak256(abi.encodePacked(_text));
    }
```



using **abi.encodePacked**

```
function hash(string memory _text, string memory _newtext)    infinite gas
public pure returns (bytes32)
{
    return keccak256(abi.encodePacked(_text, _newtext));
}
```



With abi.encodePacked

abi.encodePacked **just concatenates the raw bytes** of the strings.

So:

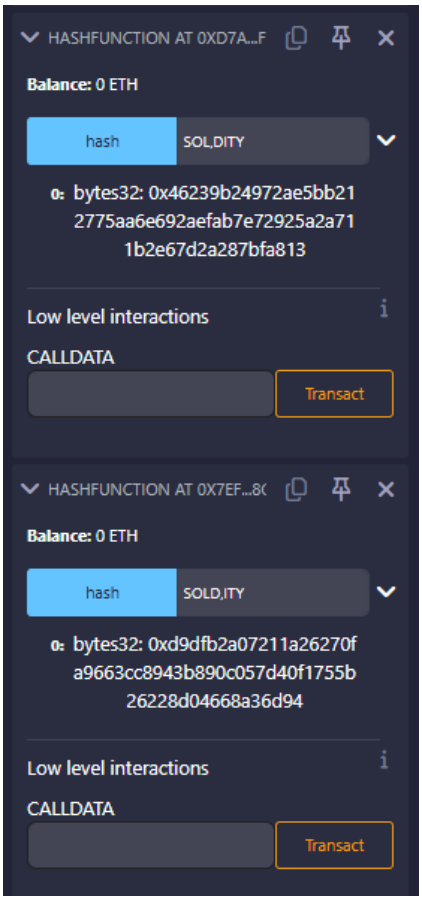
- "SOL", "DITY" → "SOLDITY"
- "SOLD", "ITY" → "SOLDITY"

Because the final byte sequence is the **same**, keccak256 hashes the same data → **same bytes32 hash**.

There is **no information** about where one string ends and the next begins.

using **abi.encode**

```
function hash(string memory _text, string memory _newtext)    infinite gas
public pure returns (bytes32)
{
    return keccak256(abi.encode(_text, _newtext));
}
```



With abi.encode

abi.encode includes **length and padding metadata** for each argument.

So internally it becomes something like:

- "SOL"(3), "DITY"(4)
- "SOLD"(4), "ITY"(3)

Even though the visible text looks similar, the encoded bytes are **structurally different**, so the hash output is **different**.

Reflection:

- abi.encodePacked is efficient but can be unsafe with multiple dynamic inputs.
- abi.encode is slightly more expensive but provides stronger guarantees against collisions.
- For cryptographic hashing and security-sensitive logic, abi.encode is generally the safer choice.